

第 10 章

外国语言测试

Si eres fluente en más de un idioma y competente probando programas de computadora, usted tiene una habilidad muy deseada en el mercado.

Wenn Sie eine zuverlässig Software Prüferin sind, und fließend eine fremde Sprache, ausser English, sprechen können, dann können Sie gut verdienen.

翻译上述西班牙文和德文的意思是：如果你是有竞争力的软件测试员，并且熟练掌握除英语之外的一门外语，你就有了很有价值的技能。

当今大多数软件发布范围是全世界，而不仅仅是某一个国家、某一种语言。Microsoft的Windows XP支持106种不同的语言和方言，从阿富汗语到匈牙利语到祖鲁语。大多数其他公司也这样做，因为意识到英语国家的市场不过是潜在客户的一小半。为在全球发布而进行软件设计和测试具有商业上的重大意义。

本章讲述如何测试为其他国家和语言编写的软件。这似乎是一个直接的过程，但是事实上不然，下面将讲述其原因。

本章重点包括：

- 为什么只进行翻译是不够的
- 单词和文本受何影响
- 足球和电话为什么重要
- 配置和兼容性问题
- 测试其他语言有多大的工作量

10.1 使文字和图片有意义

看到过按外语字面意思粗劣翻译过来的某个器械或者玩具的用户手册吗？“把第5个螺栓穿过绿杆拧紧在螺母上。”什么意思？

这就是粗劣的翻译（translation）。如果在制作外国语软件时不多花心思，则软件对于不讲英语的用户可能就是这样的感觉。逐字直译单词是容易的，但要想使整个操作提示意思明确、实用，就需要投入更多的时间和精力。

好的翻译工作者可以做到这一点，如果对两种语言都很熟练，就能够将外文翻译得读起来和原文一样。遗憾的是，在软件行业中甚至连一个像样的好翻译都找不到。

以西班牙文为例。把英语翻译成西班牙文应该是轻而易举的事，对吧？那么是指哪个国

家的西班牙文？西班牙的西班牙文吗？哥斯达黎加、秘鲁或多米尼加共和国的西班牙文呢？它们都是西班牙语国家，但是语言有极大差异，为一个国家编写的软件不能被其他国家很好地接受。即使英语也有同样的问题。不仅有美国英语，还有加拿大、澳大利亚和英国英语。在文字处理程序中可以令人惊奇地找到colour, neighbour和rumour这样的单词。

除了语言，还需要考虑地域（region或locale）——用户的国家和地理位置。使软件适应特定地域特征，照顾到语言、方言、地区习俗和文化的过程称为本地化（localization）或国际化（internationalization）。测试此类软件称为本地化测试。

10.2 翻译问题

虽然翻译只是整个本地化工作的一部分，但是从测试角度看这是重要的一环，最明显的问题是如何测试用其他语言做的产品。那么，软件测试员或者测试小组至少要对所测试的语言基本熟悉，能够驾驭软件，看懂软件显示的文字，输入必要的命令执行测试。现在也许要申请一直没时间上的斯洛文尼亚语公共大学课程了。

注意 软件测试小组一定要有人对测试的语言比较熟悉。当然，如果程序附带32种语言，难度就太大了。解决方法是委托本地化测试公司进行测试。全世界有许多这样的公司，它们几乎可以进行任何一种语言的测试。更多详情可以在因特网上查找“本地化测试”相关的主题。

不要求测试小组中的每一个人会说软件所用的当地语言，只需要有一个人会就行了。不知道单词如何说也可以检查许多内容。学会一点语言肯定会有帮助，但是从下面的讲述中可以看到没有特别流利的外语水平也可以进行许多测试工作。

10.2.1 文本扩展

可能出现的翻译问题中最直接的例子来自于文本扩展（text expansion）。虽然英语有时显得比较啰嗦，但是实践证明，当英语被翻译为其他语言，用来表达同一事物时往往需要加一些字符。图10-1显示了放置两个常用计算机用词的按钮被翻译成外国语时长度扩展的情形。一个好的大拇指规则是每个单词长度预计增加100%——例如一个按钮上；语句和短小段落长度预计增加50%——通常是在对话框和错误提示信息中的短语。

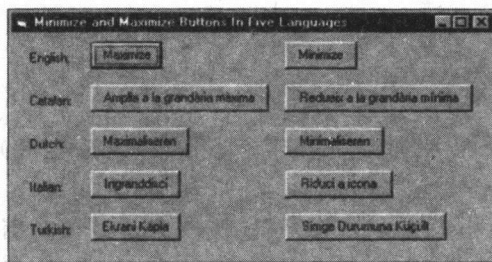


图10-1 单词Minimize和Maximize翻译成其他语言时长度发生很大的变化，迫使用户界面被重新设计以容纳它们

因为这些扩展现象，所以必须仔细测试可能受到变长了的文本影响的软件部分。要找出没有正确换行、截断的和连字符位置不对的文本，这种现象可能出现在任何地方——屏幕、

窗口、框体和按钮等。还要找到虽然文本有足够的扩展空间，但这是通过把其他的文本挤出去来实现的情况。

变长了的文本还可能导致主程序失败，甚至系统崩溃。程序员可能为英语文本信息分配了足够的内存，但是对于翻译文本字符串就不够了。软件的英文版可能工作正常，但是德文版可能在显示信息时崩溃。白盒测试员即使不认识任何外语单词，也可以发现这个问题。

10.2.2 ASCII、DBCS和Unicode

第5章“带上眼罩测试软件”简要地讨论了ASCII字符集。ASCII只能表示256种不同的字符——远不足以表示所有语言的全部字符。当开始为不同语言开发软件时，就需要找到克服该限制的解决方案。在MS-DOS时代常用的一个方法是使用称为代码页（code pages）的技术，而且今天仍然在沿用。代码页实质上是ASCII表的替换，每一种语言用一个不同的代码页。如果在法国的PC上软件在用魁北克语运行，就会读入并使用支持法文字符的代码页。俄罗斯对西里尔字符使用另一个不同的代码页，依此类推。

这个方法虽然有点笨，毕竟对少于256个字符的语言还是可行的，但是像中文、日文等包含数千个象形字符的语言就会出现问題。某些软件使用称为DBCS（双字节字符集）的系统提供对超过256个字符的语言的支持。用两个字节代替一个字节来表示最多可容纳65 536个字符。

代码页和DBCS在许多情况下已经足够了，但是会遇到一些问题，最重要的是兼容性的问题。如果在德国计算机上运行英国文字处理程序，读入一个希伯来文档，结果可能乱七八糟。没有相应的代码页或者相互之间的转换，字符就不能正确解释，甚至根本认不出来就不能解释。

解决这个麻烦的方法是使用Unicode标准。

Unicode为每一个字符提供唯一编号，

无论何种平台，

无论何种程序，

无论何种语言。

——“Unicode是什么”引自Unicode学会网站www.unicode.org

因为Unicode是由主要软件公司、硬件生产厂商和其他标准组织所支持的世界标准，所以它变得更加通用。大多数主要软件应用程序都支持它，图10-2给出了Unicode支持的多种字符。如果软件终究需要进行本地化，软件测试员和程序员就应该摆脱“古老ASCII的”的束缚，而转向Unicode，以节省时间、减少烦恼和软件缺陷。

10.2.3 热键和快捷键

英语单词Search用法语说是Rechercher。如果在软件的英语版中选择Search的热键是Alt+S，那么在法文版中需要进行改变。

在软件的本地化版本中，需要测试所有热键和快捷键工作是否正常，而且使用起来不困难——例如，需要按第3个键。同时，

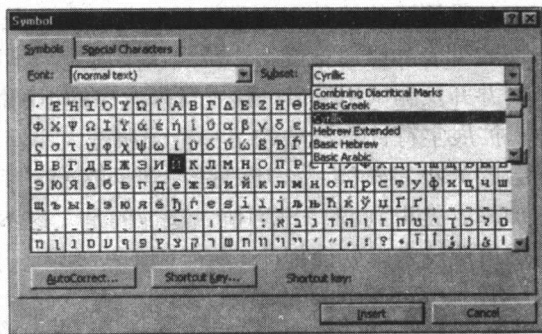


图10-2 Microsoft Word对话框显示了Unicode标准支持

不要忘记检查英文热键和快捷键是否被禁用。

10.2.4 扩展字符

本地化软件，甚至非本地化软件中存在的一个常见问题是扩展字符（extended characters）的处理。回顾古老的ASCII码表，扩展字符是指普通英文字母A~Z和a~z之外的字符。这样的例子有重音字符，例如José中的é和El Niño中的ñ；还包含了许多在键盘上见不到的象形字符，如☺♥♫等。如果软件中使用Unicode或者正确地组织代码页或者DBCS来编写，这就不成问题，但是软件测试员从来不假设，因此这值得检查。

测试扩展字符的方法是找出软件中所有接受字符输入和输出之处。在每一处都尝试使用扩展字符，看能否与常规字符一样处理。对话框、登录画面和所有文本域都是合适的对象。通过调制解调器可以收发扩展字符吗？能使用扩展字符命名文件，甚至在文件中包含扩展字符吗？它们能否正确打印？在程序之间剪切、复制和粘贴扩展字符会怎样？

技巧 测试扩展字符是否被正确处理的最简单的方法是，把它们加入测试的标准字符所在的等价划分中。和处于ASCII表边界上容易导致软件缺陷的字符一起，加进一个Æ、一个∅和一个β。

10.2.5 字符计算

与扩展字符有关的问题是软件在对其进行计算时如何解释。关于这方面的两个例子是文字排序和大小写转换。

测试的软件对文字列表排序或者按字母排列吗？也许在诸如文件名、网站地址等可选项目的列表框中进行？如果是，如何对以下文字排序？

Kopieren	Reiste
Ärmlich	Arg
Reiskorn	résumé
Reißaus	kopieën
reiten	Reisschnaps
reißen	resume

如果测试的软件在亚洲地区销售，那么是否意识到排序的依据是书写字符的笔画次序？上面的列表如果用中文写，排列次序就完全不同。要弄清楚测试的语言采用什么样的排序规则，并开发测试用例专门检查排列次序的正确性。

扩展字符计算打破的另一个领域是大小写转换。这是一个问题，因为许多程序员在学校学会的大小写转换“技巧”是在字母的ASCII值上加/减32实现大小写转换。在A的ASCII值上加32，就得到a的ASCII值。遗憾的是，这不适用于扩展字符。如果对Apple Mac扩展字符集运用该技术，就会把Ñ（ASCII值为132）转换为S（ASCII值为164），而不是ñ（ASCII值为150）——不是预期的结果。

分类和按字母顺序排列只是两个例子。仔细查看软件确定是否还有对字母或文字进行计算的其他情况，比如拼写检查？

10.2.6 从左向右和从右向左读

翻译中有一个大难题是某些语言（例如希伯来文和阿拉伯文）从右向左读，而不是从左向右读。想像一下把整个用户界面镜像翻转是什么情形。

幸好大多数主要操作系统提供了处理这些语言的内部支持。如果没有这一点，完成任务几乎是不可能的。即便如此，翻译这样的文本也不是容易的事。利用操作系统的特性来实现这些需要大量的编程工作。从测试的角度看，把它当做全新的产品，而不仅是本地化产品比较稳妥。

10.2.7 图形中的文字

另一个翻译问题是处理图形中的文字。图10-3给出了几个例子。

图10-3中的图标是选择粗体、斜体、下划线和字体颜色的标准图标。由于它们使用英文字母B、I、U和A表示，因此对不讲英文的日本人毫无意义。

它们还借助外观表达含义——B有点儿黑，I是倾斜的，U下方有一条线——但是软件不是猜谜。

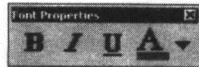


图10-3 Word 2000中有难以翻译的位图文字的一些例子

它的影响是当软件本地化时，每一个图标都要改变，以反映新的语言。如果有不少这样的图标，本地化程序就会耗资巨大。要在开发周期的早期找出图形文本软件缺陷，而不要留到最后发现。

10.2.8 让文本与代码脱离

最后要讨论的翻译问题是白盒测试问题——让文本与代码脱离。这句话的意思是说所有文本字符串、错误提示信息和其他可以翻译的内容都应该存放在与源代码独立的文件中。应该杜绝如下代码：

```
Print "Hello World"
```

大多数本地化人员不是程序员，也没有必要是。让他们修改源代码，进行语言翻译，既没有把握又有风险。他们要修改的是称为资源文件（resource file）的简单文本文件，该文件包含软件可以显示的全部信息。当软件运行时，通过查找该文件来引用信息，不管信息的内容是什么。无论信息是英文或丹麦文，都按照原文显示。

这就是说，对于白盒测试员来说，检查代码，确保没有任何嵌入的字符串未出现在外部文本文件中很重要。如果在西班牙语程序中的一个重要错误提示信息以英语的方式出现将是很令人尴尬的。

这个问题的另一个变化形式是当代码动态生成文本信息时。例如，它可能用一些文本碎片拼凑成一个大的提示信息。代码可能把以下3个字符串：

- 1) "You pressed the"
- 2) 包含用按键名称的变量字符串
- 3) "Key just in time!"

放在一起组成一条提示信息。如果变量字符串的值为stop nuclear reaction，整条信息就是：

You pressed the stop nuclear reaction key just in time!

问题在于各种语言的文字顺序是不一样的。虽然在英文中可以很好地拼在一起，但是用

中文甚至德文独立翻译每一个短语，拼在一起就会发生混乱。所以不要把字符串直接放进代码，也不要代码来连接字符串。

10.3 本地化问题

如前所述，翻译问题只是全部问题的一半。翻译文字和允许字符串包含不同字符和长度都不难。难的是修改软件使其适应国外市场。

提醒 还记得第3章“软件测试的实质”中的一些术语：精确、准确、可靠性和质量吗？

经过准确翻译和仔细测试的软件是精确和可靠的，但是如果程序员不考虑本地化的问题，程序就可能不够准确和高质量。一个软件可能外观和感觉很佳、容易理解、极其稳定，但是对于其他地方的人来说，它可能是完全错误的。首先保证产品正确地本地化，才能谈到下一步。

10.3.1 内容

如果用美国英语编写的新软件百科全书包含如图10-4所示的内容，美国人会怎么想？

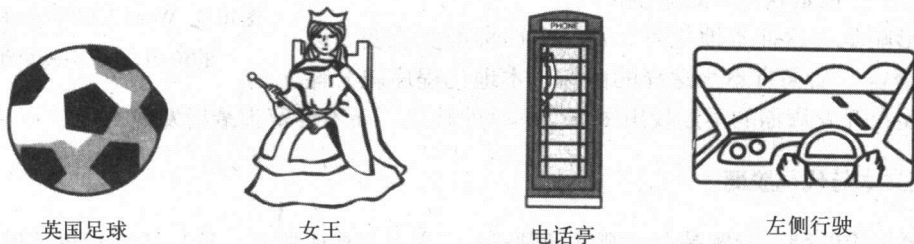


图10-4 美国百科全书中包含这些内容样本是不可思议的

在美国，soccer ball（美式足球）与football（英式足球）不是一回事！司机不能左侧行驶！这些对美国人来说可能不对，但是在其他国家则可能是千真万确的。如果测试将要本地化的产品，就需要仔细检查内容，以确保其适应使用该软件的地区。

这里所说的内容是指产品中除了代码之外的所有东西（参见第2章“软件开发的过程”）。以下清单给出了解决本地化问题要仔细审查的各类内容。不要把它当做完整清单，根据具体产品还有更多例子。考虑一下软件如果输送到其他国家还会有哪些内容可能出问题。

范例文档	图标
图片	声音
视频	帮助文件
有边界争端的地图	市场宣传材料
包装	Web链接

鼻子太长

1993年，Microsoft公司发布了两个儿童产品Creative Writer和Fine Artist。这两个产品用一个名叫McZee的助手形象来指导孩子使用软件，为选择McZee的外貌、肤色、举止和个性等，公司进行了大量研究。最后他变成了一个面目奇特的家伙，龅牙、深紫色皮肤、大鼻子。

遗憾的是，花费巨大精力绘制完这个将在屏幕上出现的动画人物之后，从

Microsoft国外办事处来了一个电话。他们曾经拿到了该软件的预览版，经过审查之后认为无法接受。原因如下：McZee的鼻子太长了。在他们的文化中，有大鼻子的人不是一般人，大鼻子总是令人与各种反面人物形象联系起来。他们说除非该产品针对该地区进行本地化，否则根本没人买。

由于为每一个市场单独创造不同的McZee形象代价太高了，因此大鼻子的McZee艺术形象被抛弃，而采用第一次为他设计的鼻子形象。

最后要说的是，软件中包含的内容无论是文字、图形、声音还是别的，都特别容易引起本地化问题。在测试有关内容的这些问题时，如果对所使用软件的地区文化不了解，一定要找一个熟悉该地区文化的人帮忙。

10.3.2 数据格式

不同的地区在诸如货币、时间和度量衡上使用不同的数据单位格式。与内容一样，这些是本地化问题，而不是翻译问题。一个用美国英语发布，使用英寸的程序不能只靠文本翻译变为使用厘米，而是需要修改代码，以改变基本的公式和网格线等。

表10-1给出了在测试本地化软件时需要熟悉的不同种类的单位。

表10-1 本地化软件的数据格式问题

单 位	问 题
度量单位	米制还是英制：米和英尺
数字	逗号、小数点或者空白分隔符；负数表示法；#号对数字的表示；1.200,00和1200.00或 -100和(100)
货币	不同的符号及其位置：30? 和? 30
日期	年月日的顺序；分隔符；前导零；长格式和短格式：dd/mm/yy和mm/dd/yy 或 May 5,2005和15 de mayo 2005
时间	12小时制还是24小时制；分隔符：3:30pm和15:30
日历	不同的日历和起始日期：在有的国家星期天不是一周的第一天
地址	行次序；使用的邮政编码：98072和T2N 0E6
电话号码	圆括号还是短线分隔符：(425)555-1212和425-555-1212和425.555.1212
纸张大小	不同的纸张和信封尺寸：美式信封和A4

幸运的是，大多数设计用于多个地区的操作系统都支持这些不同的单位及其格式，图10-5给出了一个取自Windows的例子。有了这种内置支持，程序员编写本地化软件就容易多了，但是也并非绝对安全。

注意 软件内部处理单位的方式并不需要和单位的显示方式一致。例如，在区域设置程序中的日期选项卡上显示了短日期格式m/d/yy。这并不是说操作系统只处理两位数字的年份（这样会出现千年虫问题）。在这种情况下，该设置仅意味着显示两位数字的年份，操作系统仍然支持4位数字的年份的计算。这是测试时要多考虑的一件事。

如果测试本地化软件，就需要对当地使用的度量单位非常熟悉。为了正确测试软件，需从原版软件创建的测试数据中建立不同的等价划分。

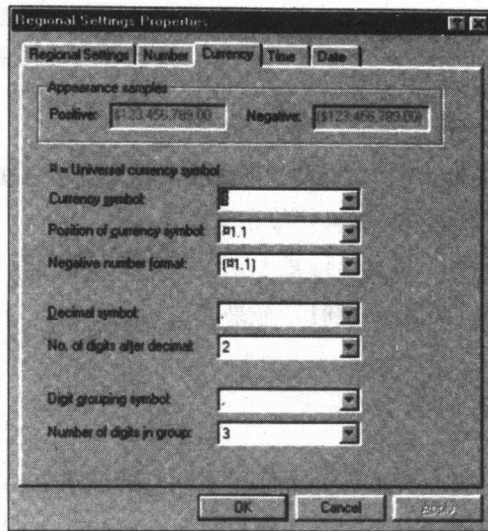


图10-5 Windows的区域设置选项允许用户选择数字、货币、时间和日期的显示方式

10.4 配置和兼容性问题

第8章“配置测试”和第9章“兼容性测试”中关于配置和兼容性测试的信息，对于软件本地化版本测试相当重要。测试软件与各种硬件和软件交互时出现的问题在遇到全新且不同的组合时会愈发扩大。此类测试不一定更加困难，只是任务量更大了。这也对寻找和获取用于测试的国外版本硬件和软件的后勤准备工作提出了更多的任务。

10.4.1 国外平台配置

Windows XP支持106种不同的语言和66种不同的键盘布局，通过Control Panel的Keyboard Properties对话框来设置，如图10-6所示。选择语言的下拉列表从阿富汗语到乌克兰语，包括美国英语之外8种版本的英语（澳大利亚、英格兰、加拿大、加勒比、爱尔兰、牙买加、新西兰和南非）、5种德语分支和20种西班牙语分支。

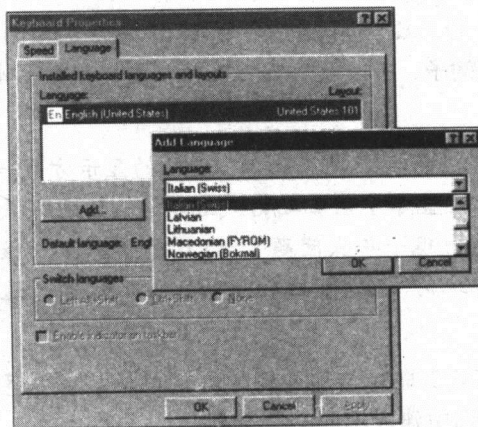


图10-6 Windows通过Keyboard Properties对话框支持使用不同的键盘和语言

图10-7给出了设计用于不同国家的三种键盘布局的示例。请注意每一种键盘都有自己语言的专用键，同时也有英文字母键。这很正常，因为英语是许多国家的第二语言，这样就允许该键盘用于同时使用当地语言和英语的软件。

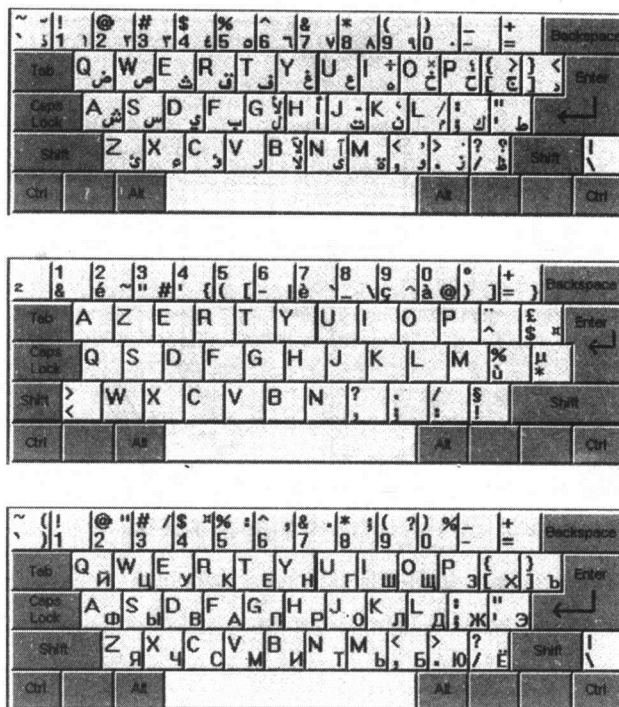


图10-7 阿拉伯、法国和俄罗斯键盘支持本国语言的专用键（www.fingertipsoft.com）

键盘也许是语言依赖性最大的硬件，但是根据测试内容，还有很多其他硬件也是如此。例如，打印机需要打印出软件发送的所有字符，并且在不同国家使用的各种规格纸张上以正确格式输出结果。如果软件使用调制解调器，就可能存在与电话线或者通信协议差异有关的问题。从根本上讲，软件可能会用到的任何外设都要在平台配置和兼容性测试的等价划分中考虑。

注意 在设计等价划分时，不要忘了应该考虑构成平台的所有硬件和软件，包括硬件本身、设备驱动程序和操作系统。在Mac机上使用法文打印机、英文操作系统以及德文版的软件，有可能是一种非常合理的用户配置。

10.4.2 数据兼容性

与平台配置测试一样，当增加了本地化问题之后，数据的兼容性测试也具有了全新的意义。图10-8说明了在两个应用程序之间转移数据可能会变得非常复杂。在该例中，使用公制单位和扩展字符的德文应用程序可以通过读写磁盘或者剪切、粘贴操作把数据移至法文程序中。然后法文应用程序可以导出数据，再导入到英文应用程序中，英文程序使用英制单位和非扩展字符，然后再把数据移回原来的德文应用程序中。

在这个循环往复的数据传输过程中，由于存在所有的度量单位和扩展字符转换和处理，因此很多地方可能有软件缺陷。其中一些软件缺陷可能源于设计上的判断，例如，在应用程

序之间移动数据时如果需要改变格式会怎样？格式会自动转换，还是会提示用户做出判断？会显示错误提示还是坚持移动数据并更改单位？

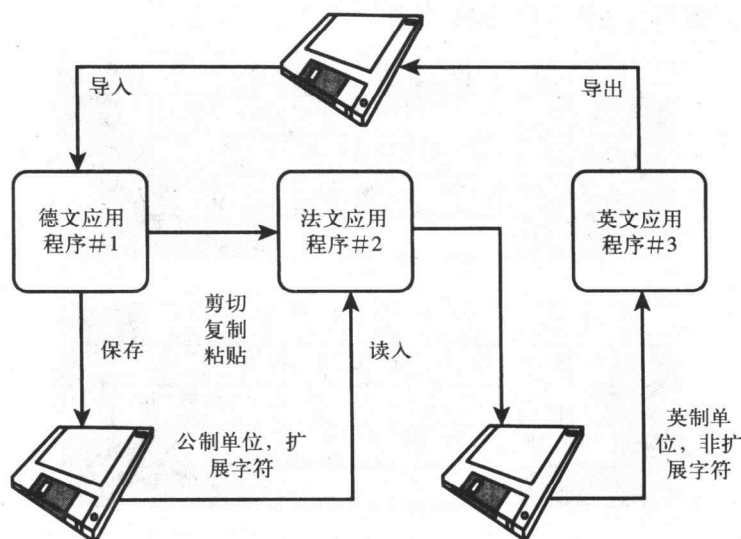


图10-8 本地化软件的数据兼容性测试可能会变得非常复杂

在测试本地化软件的兼容性之前需要回答这些重要的问题。一旦明确了这些问题，就可以正常进行兼容性测试了——只需要在等价划分中增加一些测试用例。

10.5 测试量有多大

围绕本地化测试的不确定问题是软件测试工作量究竟有多大。假如花了6个月测试美国英文版，那么测试法文本地化版本也要花6个月吗？会不会因为配置和兼容性问题增加而花费更多时间？

这个复杂的问题落到了两个问题上：

- 项目从一开始就计划本地化了吗？
- 本地化版本中更改程序代码了吗？

如果软件从一开始设计就考虑到了本章所述的问题，那么本地化版本中包含更多软件缺陷和增大测试量的风险就很小。相反，如果软件专门为美国英语市场编写，后来决定本地化为其他语言，那么明智的做法是把软件当做需要进行全部测试的全新版本。

注意 本地化测试量的要求是一个有风险的抉择，与所有的测试一样。随着测试经验的增长，就会知道决定过程中有哪些变数。

另一个问题关系到整个软件产品中什么需要改变。如果本地化工作只限于修改诸如文本和图形等内容——不是代码——测试工作可能只是对改动进行合法性检查。但是，如果因为低劣的设计或者其他问题，基本代码必须改变，就要考虑测试代码，并且检查功能和内容。

软件本地化了吗？

计划要对软件产品进行本地化的开发小组采用的一个方法是测试本地化能力（localizability）。也就是说，他们测试产品的第1个版本，假设它最终被本地化。白盒

测试员检查代码中的文本字符串、度量单位处理、扩展字符以及其他代码级问题。他们甚至会创建“伪”本地化版本。黑盒测试员仔细审查说明书和产品本身，检查诸如图形、文本和配置等本地化的问题。他们会利用“伪”版本进行兼容性测试。

最终，当产品本地化时，许多在后面才会出现的问题已经被找出并修复了，本地化工作变得比较轻松，费用也不高。

10.6 小结

Ha Ön egy rátermett és képzett softver ismer?, és folyékonyan beszél egy nyelvet az Angolon kívül, Ön egy nagyon piacképes szakképzett személy.

这段话与本章开始的一段一样——不过这回是用匈牙利文写的。看不懂没关系。从本章所述可知，了解语言只是整个本地化产品测试的一部分。通过检查产品的本地化能力和测试不依赖语言的领域，可以做很多的工作。

假如读者熟练掌握除英语之外的另一种语言，请继续阅读本书，学会在软件测试中所有可以做过的工作。随着经济全球化和计算机技术在世界范围的广泛应用，你将会像匈牙利习语中说的那样“具有相当满足市场需求的技能”。

关于Windows的本地化编程和测试的详情，请访问www.microsoft.com/globaldev。对于Mac系统，请参见Apple的网站www.developer.apple.com/intl/localization/tools.html。Linux程序员和测试员在站点www.linux.com/howtos/HOWTO-INDEX/otherlang.shtml可以找到本地化的内容。

10.7 小测验

以下是帮助读者加深理解的小测验。答案参见附录A——但是不要偷看！

1. 翻译和本地化有何区别？
2. 要了解他国语言才能测试本地化产品吗？
3. 什么是文本扩展，由此可能导致什么样的常见软件缺陷？
4. 指出扩展字符可能导致问题的一些领域。
5. 使文本字符串与代码脱离为什么重要？
6. 说出在本地化程序之间可能变化的一些数据格式类型。